ECE1778: Creative Applications for Mobile Devices

Chen Ying
Assistant Professor, Teaching Stream
Department of Electrical and Computer Engineering
University of Toronto

Last Week's Lecture

- **Styling and UI Design in React Native**
- Responsive design and theming (light/dark mode)
- Centralized styles and colors for consistency
- Advanced styling tools: Animations, Accessibility, UI libraries
- **Navigation with React Navigation**
- Native Stack Navigator: Static configuration
- Type-safe navigation (e.g., `RootStackParamList`, `NavigationProp`)

Static Configuration

createStaticNavigation + createNativeStackNavigator

App.tsx

```tsx
const RootStack = createNativeStackNavigator<RootStackParamList>({
  screens: {
    Home: HomeScreen,
    Details: DetailsScreen,
  },
});

const Navigation = createStaticNavigation(RootStack);

export default function App() {
  return <Navigation />;
}
```

Dynamic Configuration

- `createNativeStackNavigator`: Return an object with `Navigator` & `Screen` components
- `<NavigationContainer>`: Navigation state and history

App.tsx

```tsx
const Stack = createNativeStackNavigator<RootStackParamList>();

function RootStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="Details" component={DetailsScreen} />
    </Stack.Navigator>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <RootStack />
    </NavigationContainer>
  );
}
```

Example

App.tsx

```tsx
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stac
import HomeScreen from "./screens/HomeScreen";
import DetailsScreen from "./screens/DetailsScreen";
import { colors } from "./constants/colors";
import { RootStackParamList } from "./types";

const Stack = createNativeStackNavigator<RootStackParamList>();

function RootStack() {
  return (
    <Stack.Navigator
      initialRouteName="Home"
      screenOptions={{
        headerStyle: { backgroundColor: colors.primary },
        headerTintColor: colors.white,
        headerTitleAlign: "center",
        headerBackTitle: "Back",
      }}
```

Pass Props to Screens

React Navigation provides default props to screens

- `route`: Information about the current route
- `route.name`: Name of the current route (e.g., “Details”)
- `route.params`: Any data passed when navigating to this screen

```
function DetailsScreen({ route }) {
  return <Text>Received ID: {route.params.id}</Text>;
}
```

Example

Pass ID from Home screen to Details screen

- `HomeScreen`: User inputs an id and navigates to `DetailsScreen`
- `DetailsScreen`: Displays the received id

Change type definition

types.ts

```ts
export type RootStackParamList = {
  Home: undefined;
  Details: { id: string };
};
```

Live Coding

`DetailsScreen`: Displays the received id

screens/DetailsScreen.tsx

```tsx
import { View, Text } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { globalStyles } from "../styles/globalStyles";
import { NavigationProp } from "../types";
import PrimaryButton from "../components/PrimaryButton";

export default function DetailsScreen() {
  const navigation = useNavigation<NavigationProp>();

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Details Screen</Text>
      <Text style={globalStyles.text}>Received ID: </Text>
      <PrimaryButton onPress={() => navigation.navigate("Home")}>
        Go to Home
      </PrimaryButton>
    </View>
  );
}
```

DetailsScreen

screens/DetailsScreen.tsx

```tsx
import { View, Text } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { globalStyles } from "../styles/globalStyles";
import { NavigationProp } from "../types";
import PrimaryButton from "../components/PrimaryButton";

type Props = {
  route: { params: { id: string } };
};

export default function DetailsScreen({ route }: Props) {
  const navigation = useNavigation<NavigationProp>();
  const { id } = route.params;

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Details Screen</Text>
      <Text style={globalStyles.text}>Received ID: {id}</Text>
      <PrimaryButton onPress={() => navigation.navigate("Home")}>
```

Live Coding

`HomeScreen`: User inputs an id and navigates to `DetailsScreen`

screens/HomeScreen.tsx

```tsx
import { View, Text, TextInput } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { globalStyles } from "../styles/globalStyles";
import { NavigationProp } from "../types";
import PrimaryButton from "../components/PrimaryButton";

export default function HomeScreen() {
  const navigation = useNavigation<NavigationProp>();

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <TextInput
        style={globalStyles.input}
        placeholder="Enter ID"
      />
      <PrimaryButton onPress={() => navigation.navigate("Details")}>
        Go to Details
      </PrimaryButton>
```

HomeScreen

screens/HomeScreen.tsx

```tsx
import { View, Text, TextInput } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { useState } from "react";
import { globalStyles } from "../styles/globalStyles";
import { NavigationProp } from "../types";
import PrimaryButton from "../components/PrimaryButton";

export default function HomeScreen() {
  const navigation = useNavigation<NavigationProp>();
  const [id, setId] = useState("");

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <TextInput
        style={globalStyles.input}
        placeholder="Enter ID"
        value={id}
        onChangeText={setId}
```

Pass Props to Screens

React Navigation provides default props to screens

- Use `component` for default props

App.tsx

```
<Stack.Screen name="Details" component={DetailsScreen} />
```

Custom props: Use a render callback to pass

App.tsx

```
<Stack.Screen name="Details">
  {(props) => <DetailsScreen {...props} extraData={someData} />}
</Stack.Screen>
```

- `...props`: Default props are included

screens/DetailsScreen.tsx

```
export default function DetailsScreen({ route, extraData }: Props) {}
```

Example: Custom Prop

Pass `ids` between `HomeScreen` and `DetailsScreen`

- `HomeScreen`
- Displays pressable IDs from an array `ids`
- Navigates to `DetailsScreen` with selected ID
- `DetailsScreen`
- Shows the selected ID and allows deleting it
- Updates the shared `ids` array and navigates back to `HomeScreen`

Role of `App.tsx`

- Set up the stack navigator with `HomeScreen` and `DetailsScreen`
- Manage the `ids` state with `useState`
- Initializes with [“id1”, “id2”, “id3”]
- Pass `ids` and `setIds` to `HomeScreen` via `RootStack`
- Pass `setIds` to `DetailsScreen`

Live Coding

App.tsx

```tsx
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stac
import { SafeAreaProvider } from "react-native-safe-area-context";
import HomeScreen from "./screens/HomeScreen";
import DetailsScreen from "./screens/DetailsScreen";
import { colors } from "./constants/colors";
import { RootStackParamList } from "./types";

const Stack = createNativeStackNavigator<RootStackParamList>();

function RootStack() {
  return (
    <Stack.Navigator
      initialRouteName="Home"
      screenOptions={{
        headerStyle: { backgroundColor: colors.primary },
        headerTintColor: colors.white,
        headerTitleAlign: "center",
        headerBackTitle: "Back",
```

App.tsx

App.tsx

```tsx
import { useState } from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stac
import { SafeAreaProvider } from "react-native-safe-area-context";
import HomeScreen from "./screens/HomeScreen";
import DetailsScreen from "./screens/DetailsScreen";
import { colors } from "./constants/colors";
import { RootStackParamList } from "./types";

const Stack = createNativeStackNavigator<RootStackParamList>();

function RootStack({
  ids,
  setIds,
}: {
  ids: string[];
  setIds: React.Dispatch<React.SetStateAction<string[]>>;
}) {
  return (
```

`HomeScreen`

- Displays pressable IDs from `ids`
- Navigates to `DetailsScreen` with selected ID

Live Coding

screens/HomeScreen.tsx

```tsx
import { View, Text, Pressable, FlatList } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { NavigationProp } from "../types";
import { globalStyles } from "../styles/globalStyles";

type Props = {
  ids: string[];
  setIds: React.Dispatch<React.SetStateAction<string[]>>;
};

export default function HomeScreen({ ids, setIds }: Props) {
  const navigation = useNavigation<NavigationProp>();

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <FlatList
        data={ids}
        keyExtractor={(item) => item}
```

HomeScreen

screens/HomeScreen.tsx

```tsx
import { View, Text, Pressable, FlatList } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { NavigationProp } from "../types";
import { globalStyles } from "../styles/globalStyles";

type Props = {
  ids: string[];
  setIds: React.Dispatch<React.SetStateAction<string[]>>;
};

export default function HomeScreen({ ids, setIds }: Props) {
  const navigation = useNavigation<NavigationProp>();

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <FlatList
        data={ids}
        keyExtractor={(item) => item}
```

`DetailsScreen`

- Shows the selected ID and allows deleting it
- Updates the shared `ids` array and navigates back to `HomeScreen`

Live Coding

screens/DetailsScreen.tsx

```tsx
import { View, Text } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { NavigationProp } from "../types";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

type Props = {
  route: {
    params: {
      id: string;
    };
  };
};

export default function DetailsScreen({ route }: Props) {
  const navigation = useNavigation<NavigationProp>();
  const { id } = route.params;

  return (
```

DetailsScreen

screens/DetailsScreen.tsx

```tsx
import { View, Text } from "react-native";
import { useNavigation } from "@react-navigation/native";
import { NavigationProp } from "../types";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

type Props = {
  route: {
    params: {
      id: string;
    };
  };
  setIds: React.Dispatch<React.SetStateAction<string[]>>;
};

export default function DetailsScreen({ route, setIds }: Props) {
  const navigation = useNavigation<NavigationProp>();
  const { id } = route.params;

```

Pass Custom Props

In `App.tsx`, change

```
<Stack.Navigator>
  <Stack.Screen name="Home" component={HomeScreen} />
  <Stack.Screen name="Details" component={DetailsScreen} />
</Stack.Navigator>
```

to

```
<Stack.Navigator>
  <Stack.Screen name="Home">
    {(props) => <HomeScreen {...props} ids={ids} setIds={setIds} />}
  </Stack.Screen>
  <Stack.Screen name="Details">
    {(props) => <DetailsScreen {...props} setIds={setIds} />}
  </Stack.Screen>
</Stack.Navigator>
```

Pass Custom Props

screens/DetailsScreen.tsx

```tsx
type Props = {
  route: {
    params: {
      id: string;
    };
  };
  setIds: React.Dispatch<React.SetStateAction<string[]>>;
};

export default function DetailsScreen({ route, setIds }: Props) {}
```

Why not `route.params`?

screens/DetailsScreen.tsx

```
type Props = {
  route: {
    params: {
      id: string;
      setIds: React.Dispatch<React.SetStateAction<string[]>>;
    };
  };
};

export default function DetailsScreen({ route }: Props) {}
```

Navigate with

```
navigation.navigate("Details", { id, setIds });
```

Retrieve both in `DetailsScreen.tsx`

```
const { id, setIds } = route.params;
```

`route.params`

`params` is just an object you pass when navigating

- *Can* hold any serializable or non-serializable values — including functions

  But React Navigation **expects `params` to be JSON-serializable**

- Doc: **Passing parameters to routes**
- Non-serializable values can break features (e.g. persistence) or behave unexpectedly

Recommended Way

Keep `setIds` as a prop, not in `route.params`

App.tsx

```
<Stack.Screen name="Details">
  {(props) => <DetailsScreen {...props} setIds={setIds} />}
</Stack.Screen>
```

- Use `params` only for data that identifies the screen's content (like `id`)
- Pass functions, state setters, arrays as regular props

App.tsx

```
<Stack.Screen name="Home">
  {(props) => <HomeScreen {...props} ids={ids} setIds={setIds} />}
</Stack.Screen>
```

Recap: Dynamic Config

- `createNativeStackNavigator`: Returns `Navigator` & `Screen` components for dynamic setup
- `NavigationContainer`: Required root wrapper for managing navigation state, history
- Pass parameters between screens
- `navigation.navigate("Details", { id })`
- Receive parameters via `route.params` in screens
- Custom props need render callbacks to pass

Recap: React Navigation

- Native Stack Navigator: Static and dynamic configurations
- Type-safe navigation: `RootStackParamList`, `NavigationProp`, `Prop`
- Screen transitions with `useNavigation`
- Parameter passing between screen components
- `route.params`: For serializable navigation params
- Custom props: Prefer for non-serializable values like state setters

Today’s Lecture

Navigation with React Navigation

Navigation with Expo Router

Assignment 3: Navigation with Expo Router and State Management

Expo Router

File-based router for React Native and web applications

- Minimal manual navigator configuration: Routes are defined by file structure in `app/` directory
- Built on React Navigation: Leverages its power with simpler setup
- Key Benefits
- Intuitive: Mirrors web routing (e.g., Next.js)
- Type-safe: Automatic TypeScript inference for routes and params
- Universal: Works across web, iOS, Android

Expo Router: Setup

Create a new Expo app

```
npx create-expo-app@latest --template blank-typescript
```

Install dependencies

```
npx expo install expo-router react-native-safe-area-context react-native-scre
```

Setup entry point

package.json

```
{
  "main": "expo-router/entry",
}
```

- `app/_layout.tsx`: Root layout (replace `App.tsx`)

Expo Router: Setup

Add a scheme in `app.json`

app.json

```
{
  "scheme": "myapp"
}
```

- Think of the scheme as your app's custom "address"
- When someone clicks a link like `myapp://details/12`, the device knows to open this app
- Let the app jump to the right screen, even if it's closed
- Must be unique on the device so the device doesn't get confused with other apps

Rules of Expo Router

- All screens/pages are files inside of `app` directory
- First `index.tsx` is the initial route
- Root `_layout.tsx` replaces `App.tsx`
- Non-navigation components live outside of `app` directory

```
assignment-3/
├── app/
│   ├── _layout.tsx           # Root layout
│   ├── index.tsx             # "/" route (initial route)
│   ├── add-edit.tsx          # "/add-edit" route
│   └── details/
│       └── [id].tsx          # "/details/:id" route
├── components/
│   ├── ActionButton.tsx
│   ├── Card.tsx
```

Root Layout

`app/_layout.tsx`: Set up root navigator

app/_layout.tsx

```tsx
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "Home" }} />
      <Stack.Screen name="details" options={{ title: "Details" }} />
    </Stack>
  );
}
```

`<Stack>`: Defines a navigation structure

vs. React Navigation

App.tsx

```
const Stack = createNativeStackNavigator();

function RootStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeScreen} />
    </Stack.Navigator>
  );
}
```

app/_layout.tsx

```
1  import { Stack } from "expo-router";
2
3  export default function RootLayout() {
4    return (
5      <Stack>
6        <Stack.Screen name="index" options={{ title: "Home" }} />
7      </Stack>
8    );
9  }
```

Automatic Types

React Navigation

- Manually define param list types for navigation safety

types.ts

```ts
export type RootStackParamList = {
  Home: undefined;
  Details: { id: string };
};
```

Expo Router: Scans `app/` directory and generates route types automatically

- Each file = one route → type-safe navigation out of the box

Automatic Types

```
app/
 ├─ index.tsx
 ├─ details/[id].tsx
```

Expo Router will infer

```ts
// pseudo-type
type Routes = {
  "/": undefined;
  "/details/[id]": { id: string };
};
```

`router.push("/details/123")` is fully type-checked

Enable Typed Routes

Doc: **Typed Routes**

In `app.json`

app.json

```
{
  "expo": {
    "experiments": {
      "typedRoutes": true
    }
  }
}
```

Run `npx expo customize tsconfig.json`

Style Stack

app/_layout.tsx

```tsx
import { Stack } from "expo-router";
import { colors } from "../constants/colors";

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: colors.primary },
        headerTintColor: colors.white,
        headerTitleStyle: { fontWeight: "bold" },
        headerTitleAlign: "center",
        headerBackTitle: "Back",
      }}
    >
      <Stack.Screen name="index" options={{ title: "Home" }} />
      <Stack.Screen name="details" options={{ title: "Details" }} />
    </Stack>
  );
}
```

app/index.tsx

Initial route

app/index.tsx

```tsx
import { View, Text } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Home() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <PrimaryButton onPress={() => router.push("/details")}>
        Go to Details
      </PrimaryButton>
    </View>
  );
}
```

`router`

Expo Router: Navigate with URL-style paths using the global object `router`

```
import { router } from "expo-router";

router.push("/details");
```

React Navigation: Navigate with screen names using `useNavigation`

```
import { useNavigation } from "@react-navigation/native";

const navigation = useNavigation();
navigation.navigate("Details");
```

app/details.tsx

```
app/
 ├─ _layout.tsx
 ├─ index.tsx
 ├─ details.tsx
```

app/details.tsx

```tsx
import { View, Text } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Details() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Details Screen</Text>
      <PrimaryButton onPress={() => router.push("/")}>Go to Home</Primary
    </View>
  );
}
```

Pass Params

React Navigation: `route.params`

```
import { useNavigation } from "@react-navigation/native";

const navigation = useNavigation();
navigation.navigate("Details", { id });
```

Expo Router:

```
import { router } from "expo-router";

router.push({ pathname: "/details", params: { id }});
```

Recap: Expo Router Basics

- **File-based routing**: Screens are files inside `app/` directory
- **Built on React Navigation**: Simpler setup
- **Setup**: Add dependencies, set `"main": "expo-router/entry"`, define a scheme
- **Navigation structure**
- `app/_layout.tsx` defines root `<Stack>` (`App.tsx`)
- `app/index.tsx` defines initial route (`HomeScreen.tsx`)

Recap: Expo Router Basics

- **Automatic types**: Expo Router infers route + param types from file structure
- **Navigation API**: Use the global `router.push("/path")` (vs. `navigation.navigate("Name")` in React Navigation)

  Expo Router = React Navigation + file-based routing + automatic type safety

Dynamic Routes

Square brackets `[ ]` in filenames = dynamic routes

```
app/
 └── details/
      └── [id].tsx
```

- `/details/1` → opens `app/details/[id].tsx`
- `/details/abc` → also opens `app/details/[id].tsx`

  Access that parameter with `useLocalSearchParams` hook inside the page

app/details/[id].tsx

app/details/[id].tsx

```tsx
import { View, Text } from "react-native";
import { useLocalSearchParams } from "expo-router";
import { globalStyles } from "../../styles/globalStyles";

export default function Details() {
  const { id } = useLocalSearchParams();

  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Details for item {id}</Text>
    </View>
  );
}
```

`useLocalSearchParams()`: Read dynamic route parameters inside a screen

Navigate with Param

app/index.tsx

```tsx
import { View, Text } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Home() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <PrimaryButton onPress={() => router.push("/details")}>
        Go to Details
      </PrimaryButton>
      <PrimaryButton onPress={() => router.push("/details/12")}>
        Go to Details 12
      </PrimaryButton>
    </View>
  );
}
```

`useLocalSearchParams()`

```
1  import { useLocalSearchParams } from "expo-router";
2
3  export default function Details() {
4    const { id } = useLocalSearchParams();
5
6    return <Text>Details for item {id}</Text>;
7  }
```

- `[id]` in the filename → dynamic segment of the URL
- `useLocalSearchParams()` returns an object with all dynamic params
- Works with any route like `/details/12` → `{ id: "12" }`
- All param values are **strings** by default

Live Demo

app/index.tsx

```tsx
import { View, Text } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Home() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <PrimaryButton onPress={() => router.push("/details")}>
        Go to Details
      </PrimaryButton>
      <PrimaryButton onPress={() => router.push("/details/12")}>
        Go to Details 12
      </PrimaryButton>
    </View>
  );
}
```

Another Example

Dynamic Navigation with User Input

- User can type any ID in a TextInput
- Press button → navigate to the Details page with the entered ID

Live Coding

app/index.tsx

```tsx
import { View, Text } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Home() {
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <PrimaryButton onPress={() => router.push("/details")}>
        Go to Details
      </PrimaryButton>
      <PrimaryButton onPress={() => router.push("/details/12")}>
        Go to Details 12
      </PrimaryButton>
    </View>
  );
}
```

Dynamic Navigation

app/index.tsx

```tsx
import { useState } from "react";
import { View, Text, TextInput } from "react-native";
import { router } from "expo-router";
import { globalStyles } from "../styles/globalStyles";
import PrimaryButton from "../components/PrimaryButton";

export default function Home() {
  const [id, setId] = useState("");
  return (
    <View style={globalStyles.container}>
      <Text style={globalStyles.headerText}>Home Screen</Text>
      <PrimaryButton onPress={() => router.push("/details")}>
        Go to Details
      </PrimaryButton>

      <TextInput
        style={globalStyles.input}
        placeholder="Enter ID"
        value={id}
```

Recap: Dynamic Routes

- **Define dynamic routes**: Use square brackets in filenames: `app/details/[id].tsx`
- **Navigate with params**: Use `router.push("/details/12")` to open a specific route
- **Access params in the screen**: Use `useLocalSearchParams()`
- Returns an object with all dynamic segments (e.g., `{ id: "12" }`)

Lecture Summary

React Navigation

- Static/Dynamic stacks
- Type-safe params via `route.params`
- Custom props via render callbacks
- Prefer serializable data in `route.params`

Lecture Summary

Expo Router

- **File-based routing**
- Navigation: `router.push()`
- Dynamic routes: `[id]`

Speaker notes